

Ordinary Differential Equation: IVP

Part 2

Multistep methods

Multi-step methods calculate y_{i+1} at x_{i+1} from two or more previously calculated points

$$y_{i+1} = F(x_{i-2}, y_{i-2}, x_{i-1}, y_{i-1}, x_i, y_i, x_{i+1})$$

This method can be either *explicit* or *implicit*

- Explicit : Adams-Bashforth method etc..
- Implicit: Adams-Moulton method etc..

Q) Is Runge-Kutta method a single step or multistep method?

Adam-Bashforth method

It is an explicit multistep for solving a first-order ODE. It uses interpolation from the current point to previous calculated point.

Derivation for 2nd order method:

$$\frac{dy}{dx} = f(x, y) \quad \text{from } x_i \text{ to } x_{i+1}$$

$$y_{i+1} = y_i + \int_{x_i}^{x_{i+1}} f(x, y) dx$$

approximate $f(x, y)$ with a polynomial that interpolate two previous points

$$f(x, y) = f(x_i, y_i) + \frac{f(x_i, y_i) - f(x_{i-1}, y_{i-1})}{h} (x - x_i), \text{ where } h = x_i - x_{i-1}$$

$$\begin{aligned} y_{i+1} &= y_i + \int_{x_i}^{x_{i+1}} \left[f(x_i, y_i) + \frac{f(x_i, y_i) - f(x_{i-1}, y_{i-1})}{h} (x - x_i) \right] dx \\ &= y_i + \frac{h}{2} [3f(x_i, y_i) - f(x_{i-1}, y_{i-1})] \end{aligned}$$

- 2nd order Adam-Bashforth method uses two points: (x_i, y_i) and (x_{i-1}, y_{i-1})

$$y_{i+1} = y_i + \frac{h}{2} [3f(x_i, y_i) - f(x_{i-1}, y_{i-1})]$$

- 3rd order Adam-Bashforth method uses three points: (x_i, y_i) , (x_{i-1}, y_{i-1}) , (x_{i-2}, y_{i-2}) . It starts from 3rd point.

$$y_{i+1} = y_i + \frac{h}{12} [23f(x_i, y_i) - 16f(x_{i-1}, y_{i-1}) + 5f(x_{i-2}, y_{i-2})]$$

Adam-Moulton method (Implicit)

It uses $\underbrace{(x_{i+1}, y_{i+1})}_{\text{unknown}}$ to interpolate $f(x, y)$

The 2nd order method:

$$y_{i+1} = y_i + \frac{h}{2} [f(x_i, y_i) + f(x_{i+1}, y_{i+1})] \rightarrow \text{Modified Euler method (implicit)}$$

- Can be used with *Predictor-Corrector* methods
- Can be solved by a non-linear solver

Predictor – Corrector methods

It is an iterative that uses explicit(predictor) and implicit(corrector) to solve ODE. It does not need to solve complicated implicit equation directly.

Predictor : Explicit formula to estimate the solution y_{i+1}

Corrector : Use y_{i+1} (predictor) on implicit formula to obtain more accurate y_{i+1}

Algorithm flow

Given data/solution of $(x_1, y_1) \cdots (x_i, y_i)$ (previously calculated points)

1. Calculate y_{i+1} with an explicit method
2. Substitute y_{i+1} (step1) in an implicit method
3. Repeat step 2 with the refined y_{i+1} until $|y_{i+1}^{\text{new}} - y_{i+1}^{\text{old}}| \leq \varepsilon$

Example: using Euler method

Recall that the Modified Euler's method(explicit) is

- Step1: $y_{i+1}^{(1)} = y_i + f(x_i, y_i)h$ where $f(x,y)$ is known ,Explicit
 - Step2: $y_{i+1}^{(k)} = y_i + \frac{f(x_i, y_i) + f(x_{i+1}, y_{i+1}^{(k-1)})}{2}h$, $k = 2, 3, \dots$ // implicit
- until $\left| \frac{y_{i+1}^{(k)} - y_{i+1}^{(k-1)}}{y_{i+1}^{(k-1)}} \right| \leq \varepsilon$

Also, you can use Adams-Bashforth (explicit) and Adams-Moulton(implicit).

Systems of First-order ODE

We have studied ODE solver for one dependent variable $y(t)$ and one independent variable t .

There can be multiple dependent variables, e.g. $(y_1, y_2, \dots)$ for one independent variable .

Let's assume that

- Independent variable : t
- Dependent variables : $y_1, \dots, y_n$ Note: . $y_{1,1} = y_1(t_1), y_{1,2} = y_1(t_2), \dots$

A system of n first-order ODE is

$$\begin{aligned}\frac{dy_1}{dt} &= f_1(t, y_1, \dots, y_n), \quad y_1(t_1) = Y_1 \\ \frac{dy_2}{dt} &= f_2(t, y_1, \dots, y_n), \quad y_2(t_1) = Y_2 \\ &\vdots \\ \frac{dy_n}{dt} &= f_n(t, y_1, \dots, y_n), \quad y_n(t_1) = Y_n\end{aligned}$$

You can use an explicit method for each single ODEs. For each time step, the next point estimation is applied to all dependent variables.

$$\begin{aligned}t_{i+1} &= t_i + h \\y_{1,i+1} &= y_{1,i} + f_1(t, y_1, y_2, \dots, y_n)h \\&\vdots \\y_{n,i+1} &= y_{n,i} + f_n(t, y_1, y_2, \dots, y_n)h\end{aligned}$$

- **Example :** Using Euler's Explicit method

$$\begin{cases} \frac{dy}{dt} = f_1(t, y, z) \\ \frac{dz}{dt} = f_2(t, y, z) \end{cases}, \quad \text{dependent : } y, z \quad \text{indep : } t \in [a, b]$$

with the initial condition $\begin{cases} y(a) = y_0 \\ z(a) = z_0 \end{cases}$

$$\begin{aligned}t_{i+1} &= t_i + h \\y_{i+1} &= y_i + f_1(t_i, y_i, z_i)h \\z_{i+1} &= z_i + f_2(t_i, y_i, z_i)h\end{aligned}$$

- **Exercise :** Repeat using Runge-Kutta 2nd order

$$\begin{aligned}t_{i+1} &= t_i + h \\k_{y1} &= f_1(t_i, y_i, z_i) \\y_{i+1}^E &= y_i + k_{y1} \cdot h \\k_{y2} &= f_1(t_{i+1}, y_{i+1}^E, z_{i+1}) \\y_{i+1} &= y_i + (C_1 k_{y1} + C_2 k_{y2}) \cdot h\end{aligned} \quad \begin{aligned}k_{z1} &= f_2(t_i, y_i, z_i) \\z_{i+1}^E &= z_i + k_{z1} \cdot h \\k_{z2} &= f_2(t_{i+1}, y_{i+1}^E, z_{i+1}) \\z_{i+1} &= z_i + (C_1 k_{z1} + C_2 k_{z2}) \cdot h\end{aligned}$$

- **Exercise :** Use Runge-Kutta 2nd order to solve for
 $dy/dt=yz+2t$, $dz/dt=y-z^2$, $y(0)=3, z(0)=1$

$$y_0=3, z_0=1$$

$$K_1 = \left. \frac{dy}{dt} \right|_{t_i, y_0, z_0} = 3 \cdot 1 + 2t = 3 + 2t$$

$$y_1^{\varepsilon} = y_0 + K_1 \cdot h$$

$$= 3 + (3 + 2t)h$$

$$K_2 = \left. \frac{dy}{dt} \right|_{t_i, y_1, z_0} = y_1^{\varepsilon} z_0 + 2t = y_1^{\varepsilon} + 2t$$

$$y_1 = y_0 + (C_1 K_1 + C_2 K_2) \cdot h$$

$$= 3 + (C_1 \cdot (3 + (3 + 2t)h) + C_2 (y_1^{\varepsilon} + 2t))$$

$$z_1^{\varepsilon} = z_0 + K_1 h = 1 + (3 + 2t)h$$

$$z_1 = z_0 + (C_1 K_1 + C_2 K_2) \cdot h$$

$$= 1 + \{C_1 \cdot (3 + 2t) + C_2 (y_1^{\varepsilon} + 2t)\} \cdot h$$

Higher-order IVP

2nd order ODE IVP

A second order ODE with dependent variable $y(t)$ and one independent variable t

$$\frac{d^2 y}{dt^2} = f(t, y, \dot{y})$$

Example:

$$\ddot{y} + c\dot{y} + ky = u(t)$$

$$\ddot{y} = -c\dot{y} - ky + u(t)$$

$$\therefore \frac{d^2 y}{dt^2} = f(t, y, \dot{y})$$

We can write a second order ODE as a system of first-order ODEs:

$$\begin{bmatrix} \dot{y} \\ \dot{\dot{y}} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -k & -c \end{bmatrix} \begin{bmatrix} y \\ \dot{y} \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u(t)$$

$$\dot{\mathbf{x}} = \mathbf{Ax} + \mathbf{Bu}$$

Let $\frac{dy}{dt} = \dot{y} = z$

Then $\frac{d^2 y}{dt^2} = \frac{d}{dt}(z) = -c\dot{y} - ky + u(t)$

$$\frac{dy}{dt} = z(t) = f_1(t, y, z)$$

$$\dot{z} = \frac{dz}{dt} = -ky - cz + u(t)$$

A system of first-order ODE using 2nd Runge-Kutta

$$\begin{cases} \frac{dy_1}{dt} = f_1(t, y_1, y_2, \dots, y_n) \\ \frac{dy_2}{dt} = f_2(t, y_1, \dots, y_n) \\ \vdots \\ \frac{dy_n}{dt} = f_n(t, y_1, \dots, y_n) \end{cases}, \quad t \in [a, b]$$

with $y_1(a) = Y_1, \dots, y_n(a) = Y_n$

- Set $h = (b-a)/n$
- Repeat $i=1$ to n

$$\begin{aligned} K_{y1,1} &= f_1(t_i, y_{1,i}, \dots, y_{n,i}) \\ &\vdots \\ K_{yn,1} &= f_n(t_i, y_{1,i}, \dots, y_{n,i}) \\ K_{y1,2} &= f_1(t_i + h, y_{1,i} + K_{y1,1}h, \dots, y_{n,i} + K_{yn,1}h) \\ K_{y2,2} &= f_2(t_i + h, y_{1,i} + K_{y1,1}h, \dots, y_{n,i} + K_{yn,1}h) \\ &\vdots \\ K_{yn,2} &= f_n(t_i + h, y_{1,i} + K_{y1,1}h, \dots, y_{n,i} + K_{yn,1}h) \\ y_{1,i+1} &= y_{1,i} + \frac{1}{2} (K_{y1,1} + K_{y1,2})h \\ y_{n,i+1} &= y_{n,i} + \frac{1}{2} (K_{yn,1} + K_{yn,2})h \end{aligned}$$

Exercise: 2nd Runge-Kutta to solve $\ddot{y} + c\dot{y} + ky = u(t)$

$$z = f_1(t, y) = \frac{dy}{dt}$$

$$\frac{dz}{dt} = f_2(t, y, z) = -ky - cz + u(t)$$

With initial condition of $z_1 = \left. \frac{dy}{dt} \right|_{t=a}$
 $y_1 = y(a)$

Solution: Using $f_1(t_i, y_i, z_i) = z_i$, $f_2(t_i, y_i, z_i) = -ky_i - cz_i + u(t_i)$

For $i=1$ to n

$$t_{i+1} = t_i + h$$

$$K_{1,y} = f_1(t_i, y_i, z_i)$$

$$K_{1,z} = f_2(t_i, y_i, z_i)$$

$$K_{2,y} = f_1(t_{i+1}, y_i^E, z_i^E)$$

$$K_{2,z} = f_2(t_{i+1}, y_i^E, z_i^E)$$

$$y_{i+1} = y_i + \frac{1}{2} (K_{y,1} + K_{y,2})h$$

$$z_{i+1} = z_i + \frac{1}{2} (K_{z,1} + K_{z,2})h$$

End

Stability

$$\text{Total error} = \text{Truncation error} + \text{Round-off error}$$

For each step, the error should not be increased as we progress. However, in some cases the numerical error increases even $h \rightarrow 0$

Instability of the solution depends on

- Numerical method
- Size of h
- Specific differential equation

Usually explicit Runge-Kutta is stable if h is sufficiently small. In comparison, Implicit Euler method is always stable

True solution

$$\frac{dy}{dt} = -\alpha y, \quad y(0) = y(0)$$

$$y_{true}(t) = e^{-\alpha t} y(0)$$

$$y_{true}(t+h) = e^{-\alpha t} e^{-\alpha h} y(0) = y(t) e^{-\alpha h}$$

Euler method

$$y^E(t+h) = y(t) + (-\alpha y(t))h$$

$$= (1 - \alpha h) y(t)$$

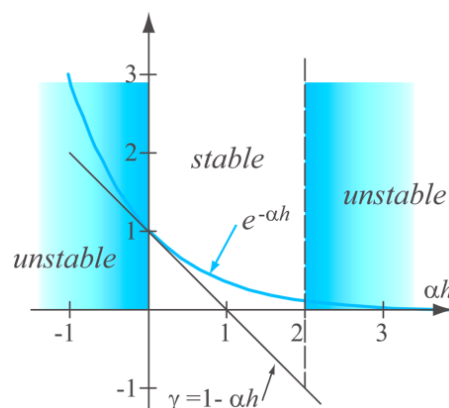
$$= k y(t)$$

$$k \cong e^{-\alpha h} = 1 - \alpha h + O(h^2)$$

$$\text{if } |k| < 1, \quad y^E(t+h) \text{ converges}$$

$$\Rightarrow |1 - \alpha h| < 1$$

$$0 < \alpha h < 2 \rightarrow \text{choose } h \text{ such that } \alpha h < 2$$



Stiff ODE

In some situation, a single solution can have several terms of different time varying rates

Example: $y(t) = e^{-at} + e^{-bt}$

One component can have a rapid rate (e^{-at}), with $|a| \gg |b|$. Then, it requires appropriate small steps. Note that the other component with a slower rate may not need the small steps.

When an explicit method is used, choosing inappropriate 'h' can give erroneous results.

- 'h' size has to do with accuracy and stability in explicit method

Example:

$$\begin{aligned}\frac{dx}{dt} &= 998x - 1998y \\ \frac{dy}{dt} &= 1000x - 2000y\end{aligned}$$

- Exact solutions are

$$\begin{aligned}x(t) &= -\frac{1998}{998} \times e^{-2t} + \frac{2996}{998} \times e^{-1000t} \\ y(t) &= -\frac{1000}{998} \times e^{-2t} + \frac{2996}{998} \times e^{-1000t}\end{aligned}$$

- The component e^{-1000t} decays much faster and becomes insignificant as t increases. Therefore, choosing not enough small 'h' may give inaccurate results for a low value of t.
- If an implicit method is used, it can give a better result for a low value of 'h'
- Euler's implicit method is unconditionally stable, more efficient to solve Stiff ODE than explicit method. In MATLAB, stiff ode solver has 's' as postfix. E.g. ode15s

	Exact	Euler explicit h=0.1	Euler implicit h=0.1
x(0.1)	-1.639	-298.8	-1.639
y(0.1)	-0.820	-298	-0.805